University of Stavanger

DAT530 - Discrete Simulation and Performance Analysis

Faculty of Science and Technology
Department of Computer Science and Electrical Engineering

Modelling the Blåsjø Hydropower Reservoir with Modular Petri Nets in GPenSIM



Written by
Parneet Kaur Dhanoa, Martin Sævareid Lauritsen
padha9311@uis.no, 267205@uis.no

4 November 2026

Abstract

Blåsjø is Norway's largest hydropower reservoir, with a storage capacity of about 3,105 Mm^3 and an estimated stored-energy capacity of 7.8 TWh. This project builds a simplified model of the reservoir as a modular, timed and coloured Petri net in GPenSIM v11. The model has three modules, Inflow, Dam and PowerStation, joined by an inter-modular connector. A free-capacity place makes the reservoir limit a P-invariant, an inhibitor arc lets water spill only when the reservoir is full, and coloured tokens record the season in which each unit of water arrived. Each week the model releases the electricity demand of that week, limited by the turbine capacity and a minimum reservoir level. We simulate 104 weeks under four scenarios: normal, wet, dry and winter turbine maintenance. In the normal year all demand is met and no water spills. The wet year spills 580 Mm^3, the dry year leaves 552 GWh of winter demand unmet because the reservoir reaches its minimum level, and winter maintenance leaves 201 GWh unmet even though the reservoir stays above 44 % full. Token colours show that by the second winter 81 % of the electricity in the normal year is made from summer water. Five verification tests, five balance and capacity checks in every run, and a reachability analysis of a reduced net support the internal consistency of the model.

Keywords: Petri net, GPenSIM, Hydropower, Blåsjø, Reservoir, Inhibitor Arc, Coloured Tokens, P-invariant, Coverability Tree

Contents

1 Introduction 4

2 Literature Review and Formulation of the Problem 4
2.1 Literature review 4
2.2 Problem definition 5
2.3 Adaptation 5

3 Method and Design 6
3.1 Overall design 6
3.2 Modular approach 6
3.3 Techniques 7
3.4 Design alternatives 8

4 Implementation 9
4.1 Programming environment 9
4.1.1 Configuration 10
4.1.2 Building and running the net 11
4.2 Modules 11
4.2.1 Inflow module 11
4.2.2 Dam module 12
4.2.3 PowerStation module 14
4.2.4 Inter-modular connector 14
4.3 Inter-modular interaction 14
4.4 Results and balance checks 15

5 Testing, Analysis and Results 16
5.1 Simple user manual 16
5.2 Sample run 16
5.3 Test data 17
5.4 Verification 18
5.4.1 Tests 18
5.4.2 Reachability analysis 19
5.5 Results 22
5.6 Analysis 23

6 Discussion 24
6.1 Originality of this work 24
6.2 Relevant works 24
6.3 Limitations 24
6.4 Further work 25
6.5 Learning experience 25

7 Conclusion 26

References 26

Appendix A 26
A1 Installation guide 26
A2 User manual 27
A3 Complete code 27

Appendix B 42

1 Introduction

This report presents a discrete-event model of a hydropower reservoir for the course DAT530 Discrete Simulation and Performance Analysis. A discrete-event dynamic system (DEDS) changes state at distinct points in time when events occur. We represent the reservoir, its inflow and its power station as a Petri net and simulate the model with the General Purpose Petri Net Simulator (GPenSIM), a MATLAB-based tool [1], [2].

The system studied is Blåsjø, the main reservoir of the Ulla-Førre hydropower complex in Rogaland, western Norway. Blåsjø holds about 3,105 Mm^3 at its highest regulated level, which corresponds to 7.8 TWh of stored energy, and its surface can vary across a range of about 125 m [4]. It is Norway's largest reservoir by stored energy, and water can be kept in it for several years [5].

Blåsjø is a useful case for this type of model because water availability and electricity demand follow opposite seasonal patterns. Most of the inflow arrives as snowmelt in late spring and summer, while demand is highest in winter. The reservoir must therefore store summer water for the winter, without overflowing in a wet year and without falling below its minimum level in a dry year. This project models these constraints with a deliberately simple release rule and studies how the reservoir behaves under normal, wet, dry and turbine-maintenance conditions.

2 Literature Review and Formulation of the Problem

2.1 Literature review

Petri nets. A Petri net is a bipartite directed graph of places and transitions connected by arcs. Tokens in places describe the state (the marking), and a transition fires when its input places hold enough tokens, moving tokens from input to output places. Murata's survey [6] remains the standard reference for Petri net properties such as boundedness, liveness and invariants, and for analysis methods such as the reachability (coverability) tree and the incidence-matrix equation.

GPenSIM. GPenSIM is a Petri net toolbox for MATLAB developed by Reggie Davidrajuh at the University of Stavanger [1], [2], [7]. It supports place/transition nets, timed nets and coloured tokens, plus analysis tools for reachability, P-invariants and T-invariants [7]. Two GPenSIM features are central to this project. First, **pre- and post-processor files** let MATLAB code decide whether an enabled transition may fire and update counters after it fires. Second, **modular Petri nets** let a large model be built from separate modules joined by an inter-modular connector (IMC), with transitions marked as ports [1].

Petri nets for hydropower. Petri nets have been applied to hydropower operations before. Ganegoda [8] built a Petri net model of the Laxapana complex in Sri Lanka (five reservoirs, five stations, 13 generators, 354.8 MW) to schedule generators and plan preventive maintenance from real-time water level, flow and generator data. That work

focuses on generator scheduling; it does not use token colours to trace the origin of stored water.

The Blåsjø reservoir. Ulla-Førre produced on average 4.45 TWh per year between 1987 and 2006 [4]. The complex also includes the Saurdal pumped-storage plant, which can lift water back into Blåsjø [4]. Storage capacity is still being extended: in June 2026 Statkraft began building the Blåfjell pump, which will lift water from three smaller lakes about 60 m up into Blåsjø and add about 54 GWh per year from December 2027 [5]. This storage role, saving summer water for winter, is what our model tries to capture.

2.2 Problem definition

The problem is to model how the reservoir stores and releases water over two years, and to see how it behaves when conditions change. Specifically, the project asks:

1. Can a Petri net guarantee, by its structure, that the reservoir never holds more than its capacity and that no water is lost or created?
2. How much water spills in a wet year, and how low does the reservoir fall in a dry year?
3. How much electricity demand is lost when the turbine runs at half capacity during winter maintenance?
4. Which season's water actually produces the winter electricity?

The problem is interesting because the constraints interact: storage capacity, the minimum level, turbine capacity and seasonal timing all limit each other. A modelling and simulation approach is suitable because these constraints are discrete (whole tokens, whole weeks) and event-driven, and because experiments on a real reservoir are impossible. The expected outcomes are a verified model and quantitative results for spill, lowest level, unmet demand and the seasonal origin of the water used.

2.3 Adaptation

The Petri net, release rule and test cases were designed for this project. The model uses GPenSIM v11 mechanisms for modular nets, coloured tokens, inhibitor arcs and processor files [1], [2]. The reservoir capacity and energy conversion factor are based on published figures for Blåsjø [4]. The weekly inflow and electricity-demand profiles are synthetic inputs constructed for the experiments; they are not measured operating data (Section 5.3).

3 Method and Design

3.1 Overall design

The system is split into three modules that follow the physical path of the water: **Inflow** (rain and snowmelt), **Dam** (the reservoir) and **PowerStation** (the turbine). A fourth element, the inter-modular connector **IMC**, holds the places shared between modules. Each module talks to the others only through its port transitions, so a module can be

changed without touching the rest. Figure 1 shows the complete Petri net with its modules, ports, shared places and the inhibitor arc.

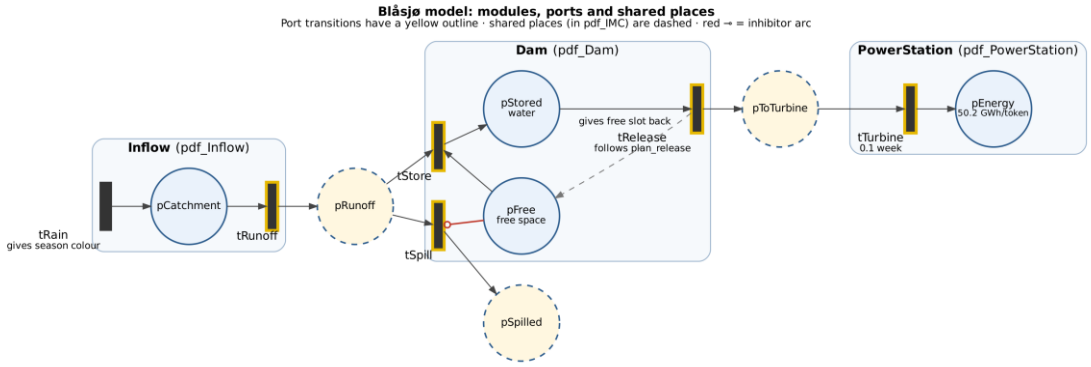


Figure 1: Petri net model of the Blåsjø system. Water tokens are made by tRain, pass the shared place pRunoff into the Dam, and leave through pToTurbine to the turbine. Port transitions have a yellow outline and the shared places in the IMC are dashed. The red arc with a circle is the inhibitor arc from pFree to tSpill.

3.2 Modular approach

Each module is defined in its own Petri net definition file (PDF) and has its own processor files. Table 1 lists the elements of each module.

Table 1: Modules of the Blåsjø Petri net.

Module	Places	Transitions	Ports	Processor files
Inflow (pdf_Inflow.m)	pCatchment	tRain, tRunoff	tRunoff	MOD_Inflow_PRE
Dam (pdf_Dam.m)	pStored, pFree	tStore, tSpill, tRelease	tStore, tSpill, tRelease	MOD_Dam_POST
PowerStation (pdf_Power-Station.m)	pEnergy	tTurbine	tTurbine	MOD_PowerStation_POST
IMC (pdf_IMC.m)	pRunoff, pSpilled, pToTurbine	none	none	COMMON_PRE (port rules)

The complete model contains seven places and six transitions. Rules that involve places shared between modules, such as storing and releasing water, are handled in COMMON_PRE. Rules that belong to one module, such as making the weekly inflow, are handled in that module's processor files.

3.3 Techniques

The model is a timed, coloured place/transition net. It uses the following techniques.

Discretisation into tokens. Water and energy are counted in whole tokens. One simulation time unit is one week, and the conversion factor follows from Blåsjø's published capacity and energy content [4]:

1 water token = 20 Mm^3; (7.8 TWh / 3,105 Mm^3) × 20 Mm^3 ≈ 50.2 GWh per token. (1)

The reservoir capacity is therefore 155 tokens (3,100 Mm^3), and the minimum operating level is 16 tokens (about 10 % full).

Complementary capacity place and P-invariant. The free space in the reservoir is modelled as its own place, pFree. Storing a token takes one token from pFree; releasing a token gives one back. This makes the capacity a structural property of the net rather than a rule in code:

$$M(pStored) + M(pFree) = 155 \text{ for every reachable marking } M \quad (2)$$

Inhibitor arc for spilling. tSpill has an inhibitor arc from pFree, so it can fire only when pFree is empty, that is, when the reservoir is full (Figure 2). When there is free space, tStore is enabled instead. Because the two transitions can never be enabled at the same time, no priorities are needed.

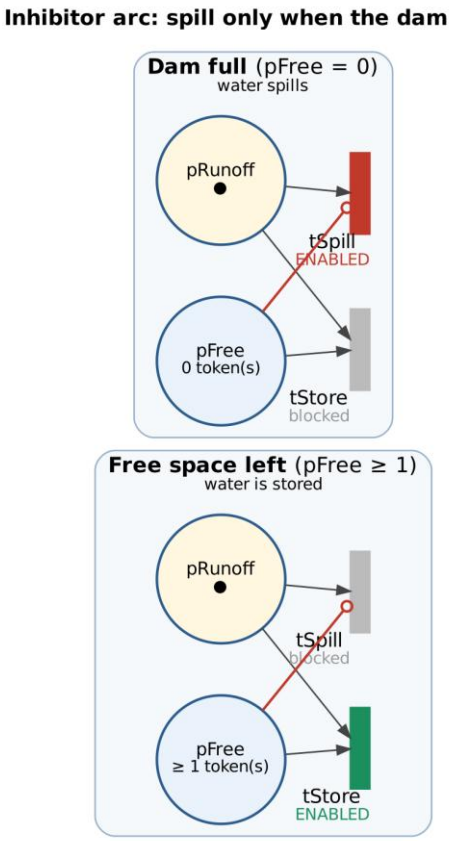


Figure 2: The inhibitor arc. With free space (pFree ≥ 1) tStore is enabled and tSpill is blocked; when the reservoir is full (pFree = 0) only tSpill can fire.

Timed transitions. The turbine transition takes 0.1 week per token and all other transitions take 0.05 week. The simulation clock step (DELTA_TIME) is 0.05, so every firing time is a whole number of clock steps.

Token colours. Each rain token is coloured with the season it arrived in (spring, summer, autumn or winter). tStore keeps that colour, tRelease always takes the oldest token first (first-in, first-out with tokenArrivedEarly), and the turbine passes the colour on to the energy token. This lets us trace which season's water produced the winter electricity (Figure 3). Water already in the reservoir at the start has no colour.

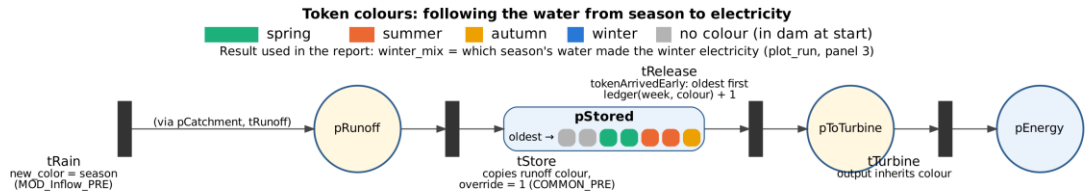


Figure 3: Token colours from season to electricity. tRain colours each token with the season, tStore keeps the colour, tRelease takes the oldest water first and records its colour in a ledger, and tTurbine passes the colour to the energy token.

Release rule. The model uses one simple rule. Each week the planned release is that week's demand, limited by the turbine capacity:

release(w) = min(demand(w), capacity(w)), capacity(w) = 4 tokens, or 2 tokens in maintenance weeks (3)

tRelease fires until the week's planned release has been sent, and never while the reservoir is at or below the minimum level. The plan is computed once, before the simulation, in dam_config.m.

3.4 Design alternatives

Table 2: Design alternatives considered.

Decision	Alternative considered	Choice and reason
Token size	1 token = 1 Mm^3	1 token = 20 Mm^3. Keeps the net to about 155 stored tokens, so runs stay fast and hand-checkable.
Capacity limit	Check ntokens('pStored') < 155 in a pre-processor	Complementary place pFree. Capacity becomes a provable P-invariant instead of a code rule.
Spill condition	A pre-processor test on the reservoir level	Inhibitor arc from pFree. The condition is visible in the net structure and in the reachability tree.
Turbines	Two turbine transitions with priorities, one of them blocked during maintenance	One turbine transition. With a firing time of 0.1 week one transition can handle the weekly release on its own, so a second turbine did not change the results. The weekly limit and

Decision	Alternative considered	Choice and reason
		maintenance are set in the release plan.
Release rule	Two policies: constant release (A) and a demand- and level-based rule (B)	One rule: release the week's demand, up to the turbine capacity. This keeps the model easy to follow and the focus on the Petri net concepts.
Structure	One flat net	Modular net with IMC. Each physical part can be tested and replaced on its own.
Water origin	One uncoloured water token type	Season colours with FIFO release. Answers which season's water feeds winter demand.

4 Implementation

This section shows how the design from Section 3 is turned into GPenSIM code. It first describes the programming environment and the configuration, then each module with its processor files, the interaction between modules, and finally how results and checks are computed. The complete code is in Appendix A3.

4.1 Programming environment

The simulation is written in MATLAB using GPenSIM v11 [1]. Each module has its own Petri net definition file (PDF), and the firing rules are placed in pre-processor (PRE) and post-processor (POST) files. Table 3 lists the files and their roles.

Table 3: Project files.

File	Role
dam_config.m	All settings: capacity, units, seasons, weekly inflow and demand, scenarios, test cases and the weekly release plan
pdf_Inflow.m, pdf_Dam.m, pdf_PowerStation.m, pdf_IMC.m	Petri net definition files for the three modules and the connector
COMMON_PRE.m	Firing rules for the port transitions tStore and tRelease
MOD_Inflow_PRE.m	Module rule: weekly rain amount and season colour
MOD_Dam_POST.m, MOD_PowerStation_POST.m	Event counters and the weekly reservoir level
week_now.m, tokens_in.m	Simulation time to week and season; tokens left in a place after a run
run_dam.m	Builds and runs the net, computes results and balance checks
main_dam.m, run_all_scenarios.m	Runs one scenario, or all four scenarios
run_tests.m, reach_pdf.m,	Verification tests, reduced net and reachability

File	Role
reachability_analysis.m	analysis
plot_run.m, plot_compare.m	Figures

4.1.1 Configuration

All model parameters are kept in one function, dam_config.m, so a scenario can be changed without touching the net. The function sets the normal values, changes them for the chosen scenario, builds the weekly plans and finally the release plan, as shown in Listing 4.1.

```
weather = 1.0;       % multiplies the inflow: wet > 1, dry < 1

if strcmp(scenario, 'S1')
    % Normal year: nothing changes.
elseif strcmp(scenario, 'S2')
    weather   = 1.4;       % 40 % more water
    cfg.START = 116;       % 116 / 155 = 75 % full at the start
elseif strcmp(scenario, 'S3')
    weather   = 0.6;       % 40 % less water
    cfg.START = 47;        % 47 / 155 = 30 % full at the start
elseif strcmp(scenario, 'S4')
    cfg.MAINT_WEEKS = 38:45;
...
end

%% 8. Make the week-by-week plans for the two years
inflow_two_years = [inflow_one_year, inflow_one_year] * weather;
cfg.plan_inflow  = diff([0, round(cumsum(inflow_two_years))]);
cfg.plan_demand = [demand_one_year, demand_one_year];

%% 10. Release plan: release the demand, up to what the turbine can use
cap = cfg.TURBINE_CAP * ones(1, cfg.WEEKS);                          % normal: 4 a week
cap(ismember(1:cfg.WEEKS, cfg.MAINT_WEEKS)) = cfg.TURBINE_CAP / 2;   % maintenance: 2
cfg.plan_release = min(cfg.plan_demand, cap);
```

Listing 4.1: Scenario settings, two-year plans and the release plan (dam_config.m, excerpt with comments shortened).

Scaling the inflow by 1.4 or 0.6 gives fractional token counts. Rounding each week on its own would be wrong, because 0.6 rounds up to 1 and the dry year would hardly be dry. Rounding the running total instead keeps the two-year total correct (146 tokens in S2 and 62 in S3). The release plan is made last because the test cases in step 9 may change the number of weeks and the demand.

4.1.2 Building and running the net

run_dam.m joins the four PDFs with pnstruct, sets the initial marking and firing times, and runs GPenSIM (Listing 4.2). The initial marking puts START tokens in pStored and the rest of the capacity in pFree, which sets up the P-invariant from Section 3.3.

```
function pni = build_petri_net(cfg)
% Three modules and their connector
pns = pnstruct({'pdf_Inflow', 'pdf_Dam', 'pdf_PowerStation', 'pdf_IMC'});

% Starting water and free space
dyn.m0 = {'pStored', cfg.START, ...
          'pFree',   cfg.CAPACITY - cfg.START};
```

```
% Firing times are in weeks.
dyn.ft = {'tTurbine',  0.1, ...
          'allothers', 0.05};

% No priorities needed: the inhibitor arc means tStore and tSpill are never
% enabled at the same time.

pni = initialdynamics(pns, dyn);
end
```

Listing 4.2: Joining the modules and setting the initial dynamics (run_dam.m).

Before the run, prepare_counters stores the configuration in global_info and sets STOP_AT to 104 weeks and DELTA_TIME to 0.05. It also creates the weekly counters (rain, stored, spilled, released, generated), the weekly reservoir level and the colour ledger used by the processor files.

week_now.m (Listing 4.3) converts GPenSIM's current time into a week number and a season. All processor files use it, so the season colour and the weekly targets always agree.

```
function [wk, season] = week_now()
global global_info

cfg = global_info.cfg;
% Convert simulation time to a week number.
wk = min(floor(current_time() + 1e-6) + 1, cfg.WEEKS);
% Get the season for that week.
season = cfg.season_of_week(mod(wk - 1, 52) + 1);
end
```

Listing 4.3: Converting simulation time to week and season (week_now.m).

4.2 Modules

4.2.1 Inflow module

The Inflow module (Listing 4.4) has a source transition tRain, the place pCatchment and the port tRunoff, which hands water to the shared place pRunoff in the IMC.

```
function [png] = pdf_Inflow()
png.PN_name = 'Inflow';

% Places in this module
png.set_of_Ps = {'pCatchment'};

% Transitions in this module
png.set_of_Ts = {'tRain', 'tRunoff'};

% Arcs: {from, to, weight}
png.set_of_As = {'tRain',      'pCatchment', 1, ...   % rain fills the catchment
                 'pCatchment', 'tRunoff',    1};      % runoff takes it away

% Ports connect this module to the others.
png.set_of_Ports = {'tRunoff'};
end
```

Listing 4.4: Inflow module definition (pdf_Inflow.m).

Since tRain has no input places, it would fire forever. MOD_Inflow_PRE (Listing 4.5) stops it once the week's inflow target is reached and gives each new token the colour of the current season. Setting override = 1 makes the new colour replace any inherited colours.

```
function [fire, transition] = MOD_Inflow_PRE(transition)
global global_info
cfg = global_info.cfg;

% Stop when this week's inflow target is reached.
if strcmp(transition.name, 'tRain')
    [wk, season] = week_now();

    fire = global_info.n.rain(wk) < cfg.plan_inflow(wk);
    if fire
        % Keep the season colour on each token.
        transition.new_color = cfg.season_names{season};   % e.g. 'summer'
        transition.override  = 1;
        global_info.n.rain(wk) = global_info.n.rain(wk) + 1;
    end
else
    fire = 1;       % tRunoff: its interface rule is in COMMON_PRE
end
end
```

Listing 4.5: Weekly inflow and season colouring (MOD_Inflow_PRE.m).

4.2.2 Dam module

The Dam module (Listing 4.6) holds the two complementary places pStored and pFree. The arcs show that storing uses a free slot and releasing returns one, and the inhibitor arc from pFree blocks tSpill while the reservoir has free space.

```
function [png] = pdf_Dam()
png.PN_name = 'Dam';

% Places in this module
png.set_of_Ps = {'pStored', 'pFree'};

% Transitions in this module
png.set_of_Ts = {'tStore', 'tSpill', 'tRelease'};

% Arcs: {from, to, weight}
png.set_of_As = { ...
    'pFree',    'tStore',   1, ...   % storing water uses one free slot ...
    'tStore',   'pStored',  1, ...   % ... and adds one token of stored water
    'pStored',  'tRelease', 1, ...   % releasing takes one token of stored water ...
    'tRelease', 'pFree',    1};      % ... and gives the free slot back

% Inhibitor arc: {place, transition, weight}
% tSpill can fire only when pFree is empty.
png.set_of_Is = {'pFree', 'tSpill', 1};

% Ports connect this module to the others.
png.set_of_Ports = {'tStore', 'tSpill', 'tRelease'};
end
```

Listing 4.6: Dam module definition with inhibitor arc (pdf_Dam.m).

The rules for the Dam module's port transitions are in COMMON_PRE (Listing 4.7). For tStore, the runoff token is selected explicitly so that the stored water keeps its season colour; without this, the uncoloured free-space token would also affect the colour. For

tRelease, the transition fires while the week's release plan has not been met and the stored level is above the minimum. Water is taken in FIFO order, and its colour is recorded in the seasonal ledger.

```
function [fire, transition] = COMMON_PRE(transition)
global global_info
cfg = global_info.cfg;
wk = week_now();
fire = 1;              % default: no extra condition (tRunoff, tSpill, tTurbine)

switch transition.name

case 'tStore'   % water enters the dam (the arc from pFree needs free space)
        % Keep the colour of the oldest runoff token.
        tok = tokenArrivedEarly('pRunoff', 1);
        transition.selected_tokens = tok(1);
        transition.new_color = get_color('pRunoff', tok(1));
        transition.override  = 1;

case 'tRelease' % water to the turbine, following the release plan
        % Follow the release plan and keep the minimum water level.
        fire = global_info.n.released(wk) < cfg.plan_release(wk) && ...
               ntokens('pStored') > cfg.MIN_LEVEL;
        if fire
            tok = tokenArrivedEarly('pStored', 1);        % oldest water first
            transition.selected_tokens = tok(1);
            % Record which season the released water came from.
            k = find(ismember(cfg.season_names, get_color('pStored', tok(1))), 1);
            if isempty(k), k = 5; end      % no colour = water already in the dam at start
            global_info.ledger(wk, k)    = global_info.ledger(wk, k) + 1;
            global_info.n.released(wk)   = global_info.n.released(wk) + 1;
        end
end
end
```

Listing 4.7: Port rules for storing and releasing water (COMMON_PRE.m, comments shortened).

After every Dam transition, MOD_Dam_POST (Listing 4.8) counts stored and spilled tokens and saves the reservoir level for the current week. The last value saved in a week is the level at the end of that week.

```
function [] = MOD_Dam_POST(transition)
global global_info
wk = week_now();

switch transition.name
    case 'tStore', global_info.n.stored(wk)  = global_info.n.stored(wk) + 1;
    case 'tSpill', global_info.n.spilled(wk) = global_info.n.spilled(wk) + 1;
end
global_info.level(wk) = ntokens('pStored');
end
```

Listing 4.8: Counting dam events and saving the weekly level (MOD_Dam_POST.m).

4.2.3 PowerStation module

The PowerStation module (Listing 4.9) has one turbine transition that takes water from pToTurbine and puts an energy token in pEnergy. GPenSIM passes the input token's colour on to the output token, so each energy token remembers the season of the water that made it. MOD_PowerStation_POST counts one energy token each time the turbine finishes.

```
function [png] = pdf_PowerStation()
png.PN_name = 'PowerStation';

% Places in this module
png.set_of_Ps = {'pEnergy'};

% Transitions in this module
png.set_of_Ts = {'tTurbine'};

% Arcs: {from, to, weight}
png.set_of_As = {'tTurbine', 'pEnergy', 1};   % the turbine makes one energy token

% The turbine takes water from outside this module.
png.set_of_Ports = {'tTurbine'};
end

function [] = MOD_PowerStation_POST(transition) %#ok<INUSD>
% Runs when the turbine finishes.
global global_info
wk = week_now();
% Count the energy made this week.
global_info.n.gen(wk) = global_info.n.gen(wk) + 1;
end
```

Listing 4.9: PowerStation module and energy counter (pdf_PowerStation.m and MOD_PowerStation_POST.m).

4.2.4 Inter-modular connector

The IMC (Listing 4.10) has no transitions of its own. It declares the three shared places and all arcs that cross module borders, so each module file only describes its own internals.

```
function [png] = pdf_IMC()
% pRunoff: water from Inflow to Dam
% pSpilled: water spilled from the dam
% pToTurbine: water sent to the turbine
png.PN_name   = 'IMC';

png.set_of_Ps = {'pRunoff', 'pSpilled', 'pToTurbine'};
png.set_of_Ts = {};

png.set_of_As = {'tRunoff', 'pRunoff', 1, ...
                 'pRunoff', 'tStore', 1,  'pRunoff', 'tSpill', 1, ...
                 'tSpill', 'pSpilled', 1, ...
                 'tRelease', 'pToTurbine', 1, ...
                 'pToTurbine', 'tTurbine', 1};
end
```

Listing 4.10: Inter-modular connector with the shared places (pdf_IMC.m).

4.3 Inter-modular interaction

Within each simulated week the modules interact in this order:

1. **Inflow:** tRain fires plan_inflow(week) times, each token coloured with the season, and tRunoff moves the tokens into the shared place pRunoff.
2. **Dam:** tStore moves each runoff token into pStored if pFree has a token; if the reservoir is full, the inhibitor arc enables tSpill and the token goes to pSpilled.

3. **Dam to PowerStation:** tRelease sends plan_release(week) tokens to pToTurbine, oldest first, while keeping the minimum level.
4. **PowerStation:** tTurbine converts each water token into an energy token in pEnergy and keeps its colour.
5. **Post-processing:** the POST files count stored, spilled and generated tokens and save the weekly reservoir level.

4.4 Results and balance checks

After the simulation, compute_results converts token counts into water and energy units (Listing 4.11). A week with no Dam event keeps the level of the week before. Demand that the generated energy did not cover is counted as unmet, and winter weeks are found from the season table so that unmet winter demand can be reported separately.

```
% Demand that the generated energy did not cover.
unmet = max(0, cfg.plan_demand - n.gen);

% Find the winter weeks (season 4).
week_of_year = mod((1:W) - 1, 52) + 1;
is_winter    = cfg.season_of_week(week_of_year) == 4;

R.inflow_mm3       = sum(n.rain) * MM3;
R.spilled_mm3      = sum(n.spilled) * MM3;
R.gen_gwh          = sum(n.gen) * GWH;
R.unmet_gwh        = sum(unmet) * GWH;
R.unmet_winter_gwh = sum(unmet(is_winter)) * GWH;
R.min_level_pct    = 100 * min(level) / cfg.CAPACITY;
R.weeks_at_min     = sum(level <= cfg.MIN_LEVEL);
```

Listing 4.11: Converting token counts to results (run_dam.m, compute_results excerpt).

Each run is checked with five conditions: the capacity bound, the capacity P-invariant, the water balance inside the reservoir, the total water balance, and the release-to-generation balance (Listing 4.12). The balances include tokens that are still in transit at the end of the run.

```
% Check the dam capacity.
check.capacity_ok = all(R.level <= cfg.CAPACITY);

% P-invariant: water + free space = capacity.
check.invariant_ok = tokens_in(sim, 'pStored') + tokens_in(sim, 'pFree') == cfg.CAPACITY;

% Water balance inside the dam.
check.dam_balance = cfg.START + sum(n.stored) - sum(n.released) ...
                    - tokens_in(sim, 'pStored');

% All incoming water must be accounted for.
check.water_balance = sum(n.rain) - sum(n.stored) - sum(n.spilled) ...
                      - tokens_in(sim, 'pCatchment') - tokens_in(sim, 'pRunoff');

% Check water released to the turbine.
check.energy_balance = sum(n.released) - sum(n.gen) - tokens_in(sim, 'pToTurbine');
```

Listing 4.12: Balance and invariant checks run after every simulation (run_dam.m, run_checks excerpt).

5 Testing, Analysis and Results

5.1 Simple user manual

Open MATLAB with the project folder as the current folder and GPenSIM v11 on the path. Then run, one at a time:

```
run_tests               % five verification tests
reachability_analysis   % coverability tree and P-invariants
main_dam                % one scenario (edit the scenario at the top)
run_all_scenarios       % all four scenarios, writes results/
```

Listing 5.1: Commands for a basic run.

5.2 Sample run

Figure 4 shows the normal scenario (S1). The reservoir level follows a seasonal cycle: it rises in summer when the snow melts and falls in winter when demand is high. It stays between about 44 % and 64 % full, well above the 10 % minimum. All 104 tokens of demand (5,221 GWh) are met, including the cold-snap peak of four tokens (about 201 GWh) in week 42. In the first winter all 31 released tokens are uncoloured water that was already in the reservoir at the start. In the second winter 25 tokens are summer water and six are spring water, so summer water makes 81 % of the winter electricity. Over the whole run the turbine used 70 uncoloured tokens, 25 summer tokens and nine spring tokens. The run takes about 8 seconds, receives 2,080 Mm^3 of inflow, spills no water and ends at 45 % full, the same level as it started.

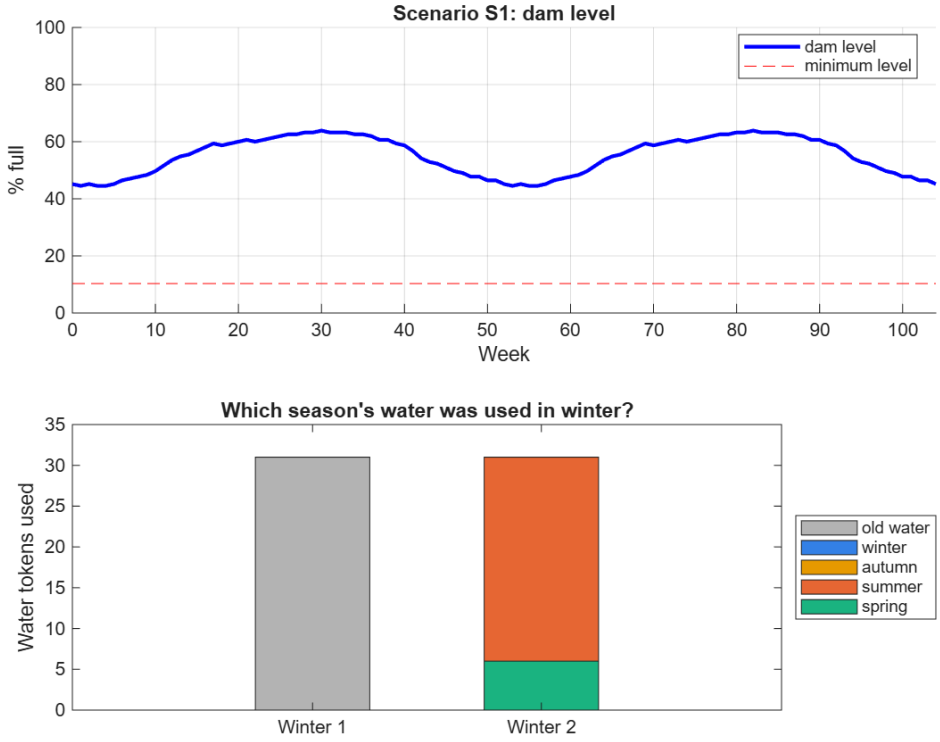


Figure 4: Scenario S1: reservoir level each week (top) and the season of the water used in each winter (bottom).

5.3 Test data

The input data are synthetic weekly profiles, built to resemble a Norwegian hydropower year that starts in early April. They are not measured data from Statkraft or NVE.

Table 4: Input data for one normal year (repeated for two years).

Quantity	Value per year	Shape
Inflow	52 tokens = 1,040 Mm^3	Peaks at 3 tokens per week in weeks 10 to 13 (June); near zero in winter
Demand	52 tokens = 2,610 GWh	31 of 52 tokens (60 %) fall in winter (weeks 36 to 52); cold snap of 3, 4, 3 tokens in weeks 41 to 43
Start level	70 tokens (45 % full)	

Inflow and demand both add up to 52 tokens per year on purpose. In the normal year the water in then equals the water out, so the reservoir neither fills up nor runs empty, and any problem in the other scenarios is caused by the scenario change rather than by the data. The synthetic annual demand is 2,610 GWh, whereas the cited historical average production for the wider Ulla-Førre complex is 4.45 TWh [4]. These figures describe different quantities, so the comparison is only contextual: the results should be read as the behaviour of the model under the chosen assumptions, not as forecasts of real production. Table 5 lists the scenarios.

Table 5: Scenarios. Each scenario changes only one or two settings compared with S1.

Scenario	Inflow	Start level	Turbine
S1 normal	× 1.0 (104 tokens in 2 years)	70 tokens (45 %)	4 tokens per week
S2 wet	× 1.4 (146 tokens)	116 tokens (75 %)	4 tokens per week
S3 dry	× 0.6 (62 tokens)	47 tokens (30 %)	4 tokens per week
S4 winter maintenance	× 1.0	70 tokens (45 %)	2 tokens per week in weeks 38 to 45 (December to January)

5.4 Verification

5.4.1 Tests

The run_tests.m script contains five tests, each with an expected result. T1 and T2 use small cases that can be checked by hand. All five tests passed (Table 6).

Table 6: Verification tests (run_tests output).

Test	Purpose	Expected	Got	Result
T1 overflow	Spill only when full	stored 3, spilled 1, free 0	stored 3, spilled 1, free 0	PASS
T2 colours, oldest first	FIFO release keeps colours	released [2 1 0 0 0], 1 left: summer	released [2 1 0 0 0], 1 left: summer	PASS
T3 normal: balances	Nothing lost or created	dam/water/energy = 0/0/0	dam/water/energy = 0/0/0	PASS
T4 wet: spill, capacity kept	Spill but never above capacity	spilled > 0, highest level ≤ 155	spilled 580 Mm^3, highest level 155	PASS
T5 dry: minimum level kept	Minimum level kept	weeks at minimum > 0, lowest level ≥ 16	weeks at minimum 8, lowest level 16	PASS

Each test follows the same pattern: run a configuration without printing, read the final marking, and compare it with the expected values. Listing 5.2 shows test T1.

```
%% T1  Overflow
% Expected: 4 tokens arrive, but the dam can hold only 3.
% Expected result: stored 3, spilled 1, free 0.
R = run_dam(dam_config('T_OVERFLOW'), false);


stored  = tokens_in(R.sim, 'pStored');
spilled = tokens_in(R.sim, 'pSpilled');
free    = tokens_in(R.sim, 'pFree');


pass = stored == 3 && spilled == 1 && free == 0 && R.check.all_ok;
got  = sprintf('stored %d, spilled %d, free %d', stored, spilled, free);
results(end+1, :) = {'T1 overflow', 'stored 3, spilled 1, free 0', got, pass};
```

Listing 5.2: Test T1, overflow (run_tests.m, excerpt with comments shortened).

In addition, every simulation run evaluates the five checks from Section 4.4. All four scenarios passed all checks (Table 7).

5.4.2 Reachability analysis

The state space of the full net is unbounded when it is analysed without the weekly firing limit that MOD_Inflow_PRE puts on tRain, because tRain is a source transition. We therefore use a reduced net (reach_pdf.m, Figure 5) to examine the same storage and spill logic with a finite supply: three rain tokens, a reservoir capacity of two, and one turbine represented by pIdle and pBusy. It keeps the complementary place pFree and the inhibitor arc to tSpill from the full model.

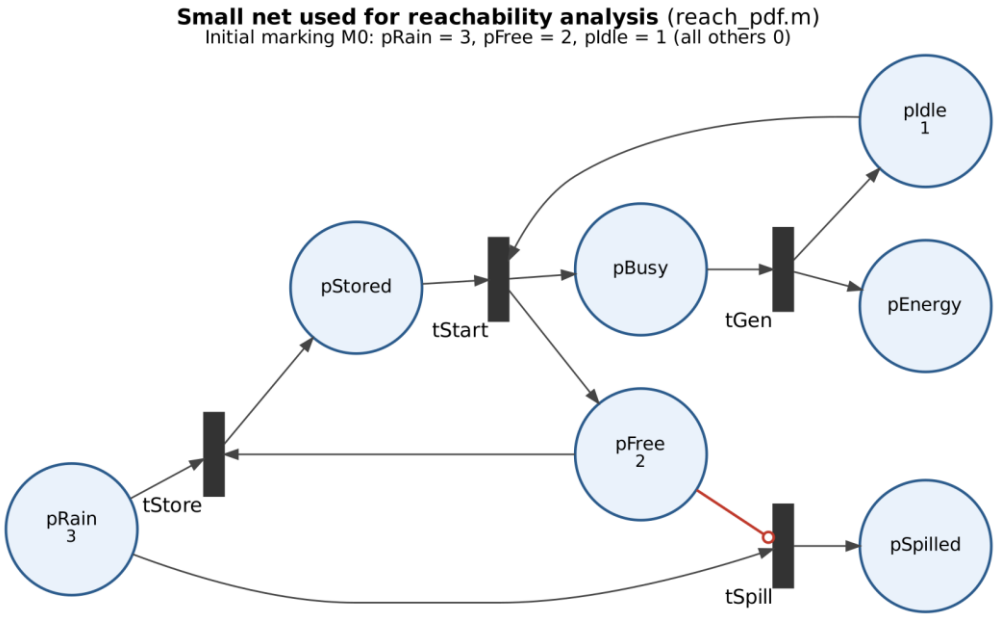


Figure 5: Reduced Petri net for the reachability analysis (reach_pdf.m). Numbers are initial tokens: 3 in pRain, 2 in pFree and 1 in pIdle.

Because the reduced net contains an inhibitor arc, the analysis uses GPenSIM's cotreei function (Listing 5.3). GPenSIM's cotree function, which can draw the tree, does not accept nets with inhibitor arcs, so the tree in Figure 6 was drawn from the cotreei printout with the same state numbers.

```
pns = pnstruct('reach_pdf');
dyn.m0 = {'pRain', 3, 'pFree', 2, 'pIdle', 1};
dyn.ft = {'allothers', 1};          % times are not used by the tree, but GPenSIM asks
pni = initialdynamics(pns, dyn);
COTREE = cotreei(pni, 500);         % coverability tree (supports inhibitor arcs)
```

Listing 5.3: Coverability tree call (reachability_analysis.m).

```
State no.: 1  ROOT node
2pFree + pIdle + 3pRain


State no.: 7    Firing event: tSpill
State: pIdle + pSpilled + 2pStored
Node type: ' '   Parent state: 4
```

```
State no.: 23   Firing event: tGen
State: 2pEnergy + 2pFree + pIdle + pSpilled
Node type: 'T'   Parent state: 20

State no.: 25   Firing event: tGen
State: 3pEnergy + 2pFree + pIdle
Node type: 'T'   Parent state: 24
```

Listing 5.4: Part of the cotreei output (states 1, 7, 23 and 25 of 25).

The tree contains 25 nodes. Five are marked D because they repeat markings already in the tree, which leaves 20 distinct reachable markings. tSpill fires only once, from state 4 to state 7, which is the only path where a rain token arrives while pFree is empty. Two nodes are terminal, marked T:

- State 25: 3pEnergy + 2pFree + pIdle. All three rain tokens were stored and turned into energy.
- State 23: 2pEnergy + 2pFree + pIdle + pSpilled. The third rain token arrived while the reservoir was full and spilled.

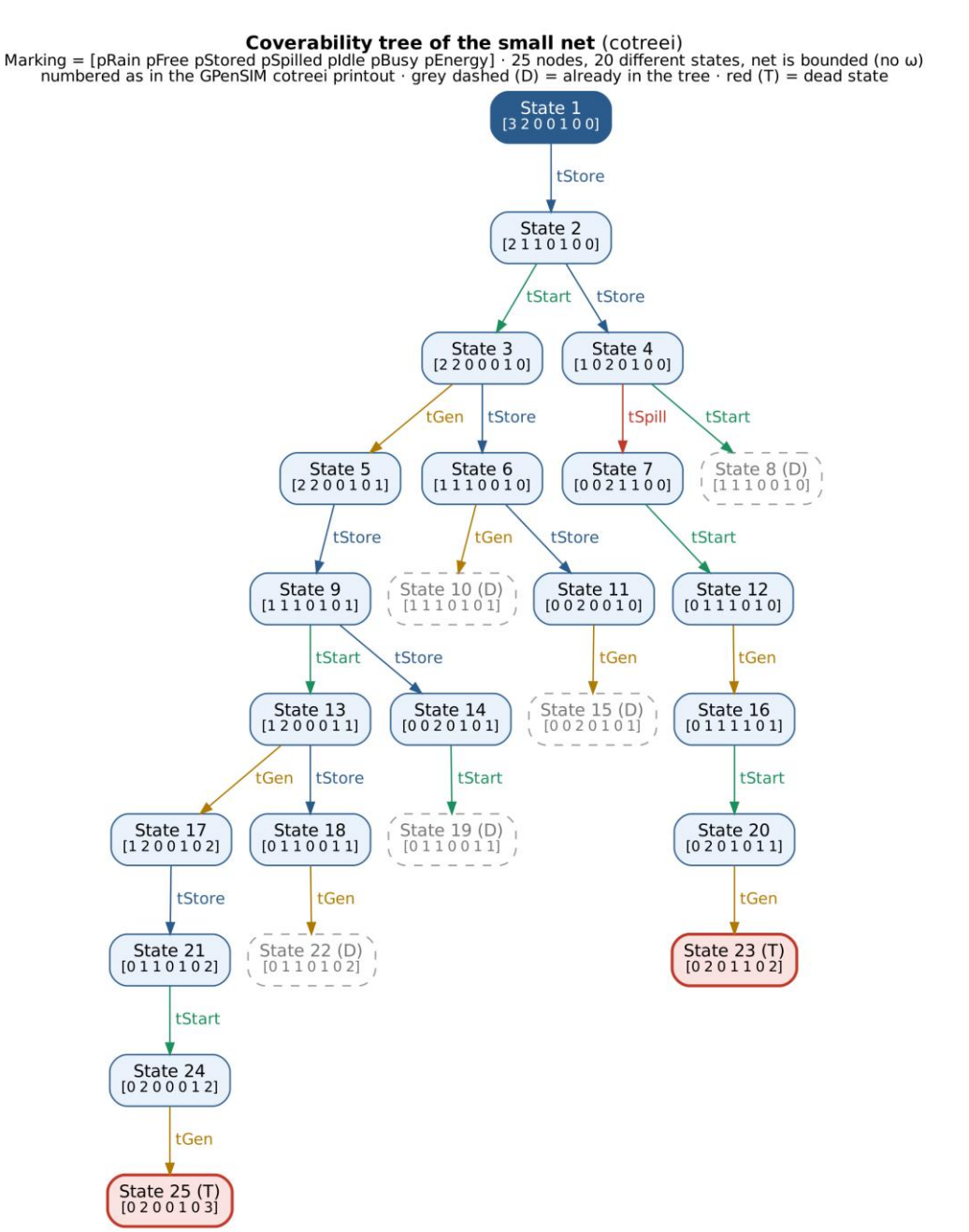


Figure 6: Coverability tree of the reduced net, numbered as in the cotreei output. Marking = [pRain pFree pStored pSpilled pIdle pBusy pEnergy]. Grey dashed nodes (D) repeat a marking already in the tree; red nodes (T) are dead states.

reachability_analysis.m checks every node of the tree. In every reachable marking pStored ≤ 2, and three P-invariants hold (Figure 7):

$$M(\text{pStored}) + M(\text{pFree}) = 2 \text{ (capacity)}$$

$$M(\text{pIdle}) + M(\text{pBusy}) = 1 \text{ (turbine)}$$

$$M(\text{pRain}) + M(\text{pStored}) + M(\text{pBusy}) + M(\text{pSpilled}) + M(\text{pEnergy}) = 3 \text{ (water)} \quad (4)$$

GPenSIM's pinvariant function finds the same three invariants. Neither terminal marking leaves water in pRain, pStored or pBusy, so the reduced model has no deadlock with

unused water. This does not prove liveness for every behaviour of the full model. For the full Blåsjø net, pinvariant finds only the capacity invariant pStored + pFree = 155. Total water is not conserved there because tRain creates water tokens and the turbine turns water tokens into energy tokens.

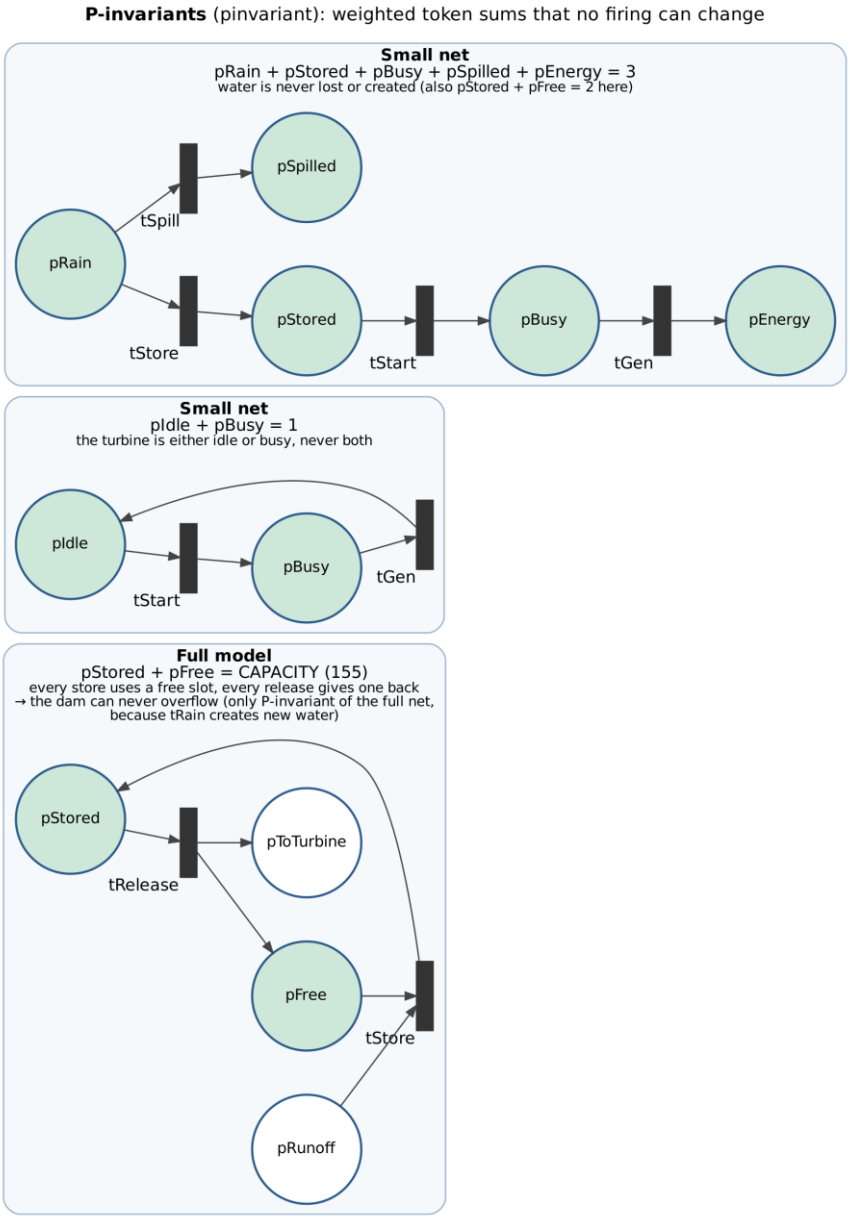


Figure 7: P-invariants of the reduced net and of the full model, shown on the parts of the nets they cover.

5.5 Results

Table 7 gives the results of the four scenarios, taken from results/scenario_results.csv. All runs passed every check. Figure 8 compares the electricity missing and the water spilled.

Table 7: Results over two simulated years. Made = electricity generated; S4 maint. = winter maintenance.

Scenario	Spilled (Mm^3)	Lowest level (%)	Weeks at min	Made (GWh)	Unmet (GWh)	Unmet in winter (GWh)	Summer share, winter 2 (%)	Checks OK
S1 normal	0	44.5	0	5,221	0	0	81	yes
S2 wet	580	74.2	0	5,221	0	0	0	yes
S3 dry	0	10.3	8	4,669	552	552	45	yes
S4 maint.	0	44.5	0	5,020	201	201	68	yes

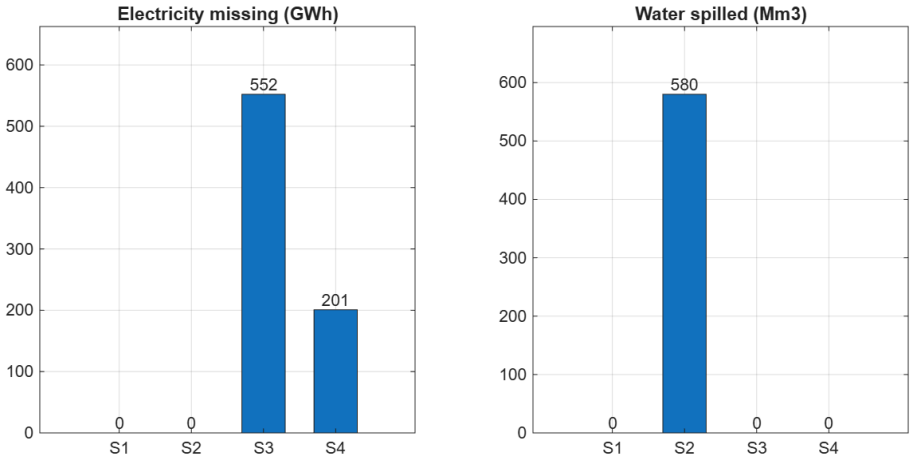


Figure 8: Electricity missing (left) and water spilled (right) in each scenario.

5.6 Analysis

Normal year (S1). Because inflow and demand are balanced, the reservoir ends where it started and all demand is met. The level never comes close to either the capacity or the minimum.

Wet year (S2). The reservoir starts 75 % full and receives 40 % more water. Since the release rule only follows demand, the extra water stays in the reservoir until it is full, and then the inhibitor arc sends 29 tokens (580 Mm^3) to pSpilled. The highest level is exactly 155 tokens, so the capacity invariant holds even when water spills. All demand is met. The summer share in winter 2 is 0 % because the reservoir starts with 116 uncoloured tokens and only 104 tokens are released over the two years, so the FIFO rule never reaches the newer, coloured water.

Dry year (S3). The reservoir starts 30 % full and receives 40 % less water. It reaches the 16-token minimum and stays there for 8 weeks, all in the second winter. Then tRelease is blocked, and 11 tokens (552 GWh) of winter demand are not delivered. The first winter is still fully supplied, but the reservoir has too little water left for the second

winter. The rule has no way to save water for later, which is the price of keeping it simple.

Winter maintenance (S4). In weeks 38 to 45 the turbine can only use 2 tokens per week. Demand above that limit in those weeks is 1 + 2 + 1 = 4 tokens, so about 201 GWh is not delivered, even though the reservoir never falls below 44 % full. Here the bottleneck is the turbine, not the water. Because less water was used, the reservoir ends at 48 % instead of 45 %.

Token colours. In S1 and S4 the old water is (almost) used up during the first winter, so the second winter runs mainly on summer water (81 % and 68 %). In S3 less summer water enters, so the second winter is a mix of spring, summer, autumn and winter water and summer provides only 45 %. The colours make the storage role of the reservoir visible: summer snowmelt is what keeps the lights on in winter.

6 Discussion

6.1 Originality of this work

The model combines three Petri net features, each with a clear purpose. The complementary place pFree makes the storage limit explicit through a P-invariant. The inhibitor arc puts the condition for spilling directly into the net structure, which also makes it visible in the coverability tree. Season-coloured tokens with FIFO release give a way to trace the seasonal origin of the water used for generation. During the project the model was also simplified on purpose: two release policies, a second turbine and transition priorities were removed after we found that they did not change the results or that the net structure already guaranteed what they were meant to do. The final model is small enough to read in full, yet still shows every technique in a measurable way.

6.2 Relevant works

Ganegoda [8] used Petri nets to model generator scheduling and maintenance across the Laxapana hydropower complex using plant data. The present project has a narrower scope: one reservoir and one turbine. Its focus is instead on structural guarantees (the capacity invariant and the inhibitor arc), on colour-based tracing of water origin, and on how inflow and maintenance conditions affect spill and unmet demand. The S4 experiment shows, for the synthetic profile used here, that maintenance during a high-demand winter period loses electricity even when there is enough water.

6.3 Limitations

- **Synthetic data.** The inflow and demand profiles are constructed inputs, not measured time series. The assumed annual demand (2,610 GWh) is lower than the cited historical average production of the wider Ulla-Førre complex (4.45 TWh) [4]. These are different measures, so the comparison is contextual rather than a validation.

- **Simple release rule.** The release only follows the week's demand. It does not release extra water when the reservoir is nearly full (S2 spills) and does not save water in a dry year (S3 runs out in the second winter).
- **Deterministic model.** Each scenario is one fixed run, so the results have no spread or confidence intervals. Real inflow is uncertain.
- **Coarse resolution.** One token is 20 Mm^3 (about 50 GWh) and one time step is one week, so small differences are rounded away.
- **Simplified physics.** Energy per token is constant, although real output depends on the head (water level). Pumping (Saurdal, Blåfjell) and the other reservoirs in Ulla-Førre are not modelled.
- **Perfect knowledge of demand.** The release plan uses each week's demand exactly; a real operator works from forecasts.
- **FIFO colouring.** Oldest-water-first is an accounting convention; real reservoir water mixes.

6.4 Further work

1. Add a level-based release rule, for example releasing extra water above 85 % full and saving water below 30 % full, and compare it with the current rule on spill and unmet winter demand.
2. Replace the synthetic profiles with measured inflow series (for example from NVE) and run each scenario many times with random inflow to get distributions of unmet demand and spill.
3. Model pumped storage as a transition from a lower reservoir back to pStored, and extend the net to the other Ulla-Førre reservoirs as further modules joined by the IMC.
4. Make energy per token depend on the reservoir level to represent head.

A good starting point is dam_config.m, where every parameter, scenario and the release plan are set.

6.5 Learning experience

The modular structure made the model easier to organise and test. The small cases T1 and T2 could be checked by hand before running the two-year scenarios. Handling token colours was one of the harder details: GPenSIM combines input-token colours by default, so tStore had to select the runoff token and override the resulting colour, otherwise the uncoloured free-space token could affect the colour of the stored water. We also learned that a simpler model is often a better model. Our first version had two release policies, two turbines, priorities and extra checks. Reviewing the code showed that the second turbine had no effect, the priorities were not needed because of the inhibitor arc, and one of the checks could never fail. Removing these parts made the model shorter and easier to explain without losing any of the Petri net concepts. Finally, the reachability analysis taught us that a source transition makes the full state space unbounded, so a

reduced net with the same storage and spill logic was needed, and that GPenSIM's cotree cannot be used on nets with inhibitor arcs, so cotreei had to be used instead.

7 Conclusion

This project developed a modular, timed and coloured Petri net of the Blåsjø reservoir in GPenSIM. The Inflow, Dam and PowerStation modules are joined by an inter-modular connector. The model guarantees the reservoir capacity through the P-invariant pStored + pFree = 155 and lets water spill only through an inhibitor arc. Five tests, five checks in every run, and a reachability analysis of a reduced net with 20 distinct states and no deadlock support the internal consistency of the implementation. Over 104 simulated weeks the normal year met all demand without spill. The wet year spilled 580 Mm^3 while the capacity was never exceeded, the dry year left 552 GWh of winter demand unmet when the reservoir reached its minimum level, and winter turbine maintenance left 201 GWh unmet although enough water was stored. Token colours showed that in the normal year 81 % of the second winter's electricity came from summer water. These results show how a few Petri net techniques can turn the main constraints of a hydropower reservoir into a model that is both simple and verifiable.

References

1. R. Davidrajuh, *Petri Nets for Modeling Complex Discrete-Event Systems: An Approach Based on GPenSIM*. Springer, 2021. MathWorks book page
2. R. Davidrajuh, *Modeling Discrete-Event Systems with GPenSIM: An Introduction*. Springer, 2018. OverDrive listing
3. R. Davidrajuh, "Formatting Your MID280 Project Report," University of Stavanger, rev. August 10, 2013 (course handout).
4. Wikipedia, "Ulla-Førre," accessed November 3, 2026. Ulla-Førre
5. Statkraft, "Statkraft to pump more water into Blåsjø reservoir," June 15, 2026. Statkraft news
6. T. Murata, "Petri nets: Properties, analysis and applications," *Proceedings of the IEEE*, vol. 77, no. 4, pp. 541-580, 1989.
7. University of Hamburg, Petri Nets World tool database, "GPenSIM." GPenSIM entry
8. D. T. Ganegoda, "Network implementation Petri net model for Laxapana power plant complex," MSc thesis, University of Moratuwa, 2019. Repository record

Appendix A

A1 Installation guide

1. Install MATLAB (any recent release that supports GPenSIM v11).

2. Download GPenSIM v11 and add its folder to the MATLAB path (addpath(genpath(‘’))).
3. Unzip the project folder Blasjo_Simple_Readable and set it as MATLAB’s current folder. No other toolboxes are needed.

A2 User manual

- **One scenario:** open main_dam.m, set scenario to ‘S1’, ‘S2’, ‘S3’ or ‘S4’, then run it. It prints a summary and the colour count of the released water, and saves results/run_.png.
- **All scenarios:** run run_all_scenarios. It prints Table 7 and writes results/scenario_results.csv, results/compare_scenarios.png and results/all_results.mat.
- **Verification:** run run_tests (prints PASS or FAIL for each test) and reachability_analysis (prints the coverability tree, the invariant checks and the P-invariants).
- **Changing assumptions:** edit dam_config.m (capacity, minimum level, start level, inflow and demand profiles, turbine capacity, maintenance weeks).
- **Plot files:** plot_run and plot_compare are called by main_dam and run_all_scenarios. They need results as input and cannot be run on their own.

Output of main_dam for scenario S1:

```
=== S1 (104 weeks, 8 s) ===
Water:  inflow 2080 Mm3, spilled 0 Mm3
Level:  start 45 %, lowest 45 %, end 45 %, weeks at minimum 0
Energy: generated 5221 GWh, unmet 0 GWh (winter 0)
Water used in winter 1 / winter 2 [spring summer autumn winter no-colour]:
        [0 0 0 0 31]  /  [6 25 0 0 0]   -> summer share in winter 2: 81 %
Checks: capacity 1, invariant 1, balances 0/0/0  -> ALL OK


Water tokens released, by colour:
  spring                                 9
  summer                                25
  autumn                                 0
  winter                                 0
  no colour (already in dam at start)   70
```

A3 Complete code

The project consists of 19 MATLAB files (Table 3). They are listed in full below.

```
function cfg = dam_config(scenario)
% DAM_CONFIG  All settings and numbers used by the Blasjo dam model.
%
% Example:  cfg = dam_config('S1')
%
% This file does not run the Petri net. It only fills the struct "cfg".
% Every other file (run_dam, COMMON_PRE, MOD_* files, plots) reads its
% numbers from cfg, so all assumptions of the model are in one place.
%
% Scenarios:
%   S1  normal year (the reference case)
%   S2  wet year: 40 % more water, dam starts 75 % full
%   S3  dry year: 40 % less water, dam starts 30 % full
%   S4  turbine maintenance in winter: half capacity in weeks 38-45
%   'T_OVERFLOW', 'T_COLOUR'  tiny test cases used by run_tests.m
%
% Units used everywhere in the model:
```

```
%   1 time unit     = 1 week
%   1 water token   = 20 million m3 (Mm3) of water
%   1 water token through the turbine = 50.2 GWh of electricity
%
% The order of the steps below matters:
%   steps 1-6  normal settings
%   step 7     the scenario changes some of them
%   step 8     the weekly plans are made from the (changed) settings
%   step 9     test cases overwrite the plans with tiny numbers
%   step 10    the release plan is made last, from the final settings

cfg.scenario = scenario;    % saved so results and plots can show the scenario



%% 1. The dam ------------------------------------------------------------
% CAPACITY: 155 tokens x 20 Mm3 = 3,100 Mm3, about the real Blasjo
% reservoir (about 3,105 Mm3). In the Petri net, the dam has one token in
% pFree for every empty slot, so this number also gives the P-invariant
%     pStored + pFree = CAPACITY
% which is why the dam can never hold more than 155 tokens.
cfg.CAPACITY  = 155;

% MIN_LEVEL: 16 / 155 = about 10 % full. At or below this level no water
% is sent to the turbine (checked in COMMON_PRE for tRelease), like the
% lowest level a real reservoir is allowed to reach.
cfg.MIN_LEVEL = 16;

% START: water in the dam at the start of week 1 (early April).
% 70 / 155 = 45 % full, a normal level before the snow melts.
% This water has no colour, because we do not know which season it came from.
cfg.START     = 70;

% WEEKS: two years. In year 1 the turbine mostly uses the old water with no
% colour. By year 2 that water is used up, so the winter of year 2 shows
% which season's water really makes the winter electricity
% (winter_mix in run_dam compares winter 1 and winter 2).
cfg.WEEKS     = 104;



%% 2. Units --------------------------------------------------------------
% Used only to turn token counts back into real numbers in the results.
% GWH_PER_TOKEN: the real plant makes about 7.8 TWh from about 3,105 Mm3,
%   7,800 GWh / 3,105 Mm3 x 20 Mm3 = about 50.2 GWh per token.
cfg.MM3_PER_TOKEN = 20;
cfg.GWH_PER_TOKEN = 50.2;



%% 3. Seasons ------------------------------------------------------------
% Week 1 is early April, so the year starts just before the snow melts.
% Season number for each week of the year:
%   weeks  1-9   spring (Apr-May)   -> 1    ( 9 weeks)
%   weeks 10-22  summer (Jun-Aug)   -> 2    (13 weeks)
%   weeks 23-35  autumn (Sep-Nov)   -> 3    (13 weeks)
%   weeks 36-52  winter (Dec-Mar)   -> 4    (17 weeks)
% Winter is the longest because December to March is the cold period
% with little inflow and high demand. 9 + 13 + 13 + 17 = 52 weeks.
%
% season_names are also the token colours: tRain (MOD_Inflow_PRE) gives
% each new water token the name of the current season as its colour.
cfg.season_names = {'spring', 'summer', 'autumn', 'winter'};

spring = 1 * ones(1, 9);
summer = 2 * ones(1, 13);
autumn = 3 * ones(1, 13);
winter = 4 * ones(1, 17);
cfg.season_of_week = [spring, summer, autumn, winter];   % 52 values, one per week
```

```
%% 4. Water arriving each week in a normal year (tokens) -----------------
% One value per week, in the same order as the seasons above.
% Most water comes from melting snow in May and June (3 tokens a week).
% There is almost no inflow in winter, because the water is frozen as snow.
% Total for one year: 52 tokens = 1,040 Mm3.
inflow_one_year = [0 1 0 1 1 2 1 2 1 ...                % spring
                   3 3 3 3 2 2 2 2 1 1 1 1 1 ...        % summer
                   1 2 1 2 1 1 1 2 0 1 1 0 1 ...        % autumn
                   0 0 1 0 0 0 0 1 0 0 0 0 0 1 0 1 0];  % winter



%% 5. Electricity needed each week (energy tokens) ----------------------
% One value per week. This is the opposite pattern to the inflow:
% low demand in summer, high demand in winter (heating).
% Weeks 41-43 are a cold snap with the highest demand (3, 4, 3 tokens).
%
% Total for one year: 52 tokens, exactly the same as the inflow.
% This is on purpose: in a normal year (S1) the water in equals the water
% out, so the dam neither fills up nor runs empty. Any problem in S2-S4 is
% then caused by the scenario change, not by the data.
% 31 of the 52 tokens are needed in winter.
demand_one_year = [1 0 1 1 0 0 0 1 0 ...                % spring
                   1 0 0 1 1 0 0 0 2 0 0 0 2 ...        % summer
                   0 1 0 1 1 0 1 1 1 1 1 1 1 ...        % autumn
                   1 2 1 2 1 3 4 3 1 2 2 1 2 1 2 1 2];  % winter



%% 6. Turbine -------------------------------------------------------------
% TURBINE_CAP: the most water tokens the turbine can use in one week.
% The highest weekly demand is 4 (the cold snap), so with 4 the turbine can
% always meet the demand when it works normally.
cfg.TURBINE_CAP = 4;

% MAINT_WEEKS: weeks with maintenance, when the turbine works at half
% capacity (2 tokens a week). Empty = no maintenance. Only S4 sets this.
cfg.MAINT_WEEKS = [];



%% 7. Change the settings for the chosen scenario -------------------------
% Each scenario changes only one or two settings compared with S1.
% So any difference in the results comes from that change alone.

weather = 1.0;       % multiplies the inflow: wet > 1, dry < 1

if strcmp(scenario, 'S1')
    % Normal year: nothing changes. All other scenarios are compared to this.

elseif strcmp(scenario, 'S2')
    % Wet year: tests that the dam spills when full and never goes above
    % CAPACITY (test T4 in run_tests.m).
    weather   = 1.4;       % 40 % more water
    cfg.START = 116;       % 116 / 155 = 75 % full at the start

elseif strcmp(scenario, 'S3')
    % Dry year: tests that the minimum level is kept (test T5).
    weather   = 0.6;       % 40 % less water
    cfg.START = 47;        % 47 / 155 = 30 % full at the start

elseif strcmp(scenario, 'S4')
    % Maintenance in December-January, which covers the cold snap.
    % In weeks 38-45 the turbine can only use 2 tokens a week. Demand above 2
    % in those weeks is 1 + 2 + 1 = 4 tokens, so 4 x 50.2 = about 201 GWh
    % cannot be delivered. Weeks 38-45 are only in year 1 (weeks 1-104).
    cfg.MAINT_WEEKS = 38:45;

elseif strcmp(scenario, 'T_OVERFLOW') || strcmp(scenario, 'T_COLOUR')
    % Tiny test cases: allowed here, but set up later in step 9,
    % because they must overwrite the plans made in step 8.
```

```
else
    % Stop with a clear message if the scenario name is misspelled,
    % instead of quietly running the normal year.
    error('Unknown scenario %s', scenario);
end



%% 8. Make the week-by-week plans for the two years ------------------------
% The same one-year pattern is used for both years, multiplied by weather.
inflow_two_years = [inflow_one_year, inflow_one_year] * weather;

% With weather 1.4 or 0.6 the weekly values become fractions (e.g. 0.6),
% but tokens must be whole numbers.
% Rounding each week on its own would be wrong: 0.6 rounds up to 1, so the
% dry year would hardly be dry (78 tokens instead of 62).
% Instead we round the running total and take the weekly differences.
% Then each week is a whole number, and the total stays correct.
cfg.plan_inflow  = diff([0, round(cumsum(inflow_two_years))]);

% Demand does not depend on the weather: both years are the same.
cfg.plan_demand = [demand_one_year, demand_one_year];



%% 9. Tiny test cases (numbers small enough to check by hand) -----------
% These come after step 8 on purpose: they overwrite the normal plans.
% MIN_LEVEL = 0 so that even 1-3 tokens can be released.

if strcmp(scenario, 'T_OVERFLOW')
    % Test T1: capacity 3, but 4 tokens arrive and none are released.
    % Expected by hand: 3 stored, 1 spilled, 0 free space.
    % This shows that the inhibitor arc lets tSpill fire only when pFree = 0.
    cfg.WEEKS       = 2;
    cfg.CAPACITY    = 3;
    cfg.START       = 0;
    cfg.MIN_LEVEL   = 0;
    cfg.plan_inflow = [4 0];
    cfg.plan_demand = [0 0];
end

if strcmp(scenario, 'T_COLOUR')
    % Test T2: two spring tokens arrive in week 1 and two summer tokens in
    % week 10. Three tokens are released in week 12.
    % If the oldest water goes first (tokenArrivedEarly in COMMON_PRE),
    % 2 spring + 1 summer tokens are released and 1 summer token stays.
    % This shows that colours are kept and the order is oldest first.
    cfg.WEEKS       = 12;
    cfg.CAPACITY    = 10;
    cfg.START       = 0;
    cfg.MIN_LEVEL   = 0;
    cfg.plan_inflow = zeros(1, 12);
    cfg.plan_inflow(1)  = 2;
    cfg.plan_inflow(10) = 2;
    cfg.plan_demand = zeros(1, 12);
    cfg.plan_demand(12) = 3;
end



%% 10. Release plan: release the demand, up to what the turbine can use -----
% This is the whole release rule of the model:
%   release this week's demand,
%   but not more than the turbine can use (half during maintenance).
% It is made last because it uses the final WEEKS, plan_demand and
% MAINT_WEEKS, which the test cases in step 9 may have changed.
%
% COMMON_PRE fires tRelease until plan_release(week) tokens have gone out
% that week, or until the dam reaches MIN_LEVEL.
cap = cfg.TURBINE_CAP * ones(1, cfg.WEEKS);                          % normal: 4 a week
cap(ismember(1:cfg.WEEKS, cfg.MAINT_WEEKS)) = cfg.TURBINE_CAP / 2;   % maintenance: 2
```

```
cfg.plan_release = min(cfg.plan_demand, cap);
end
```

Listing A.1: dam_config.m

```
function R = run_dam(cfg, verbose)
% Build and run the Petri net, then collect the results.
%
% Example: R = run_dam(dam_config('S1'))
% Use false as the second input to hide the summary.
%
% Main steps:
% 1. Set up counters.
% 2. Join the modules and set the starting tokens and times.
% 3. Run GPenSIM.
% 4. Calculate the results.
% 5. Check the balances and capacity.

if nargin < 2
    verbose = true;
end

global global_info PN
global_info = struct();    % forget everything from a previous run
PN = [];

% Step 1
prepare_counters(cfg);

% Step 2
pni = build_petri_net(cfg);

% Step 3
tic;
sim = gpensim(pni);
R.runtime_s = toc;

% Step 4
R = compute_results(R, cfg, sim, global_info);

% Step 5
R.check = run_checks(cfg, sim, global_info, R);

if verbose
    print_summary(R);
end
end



%% ========================================================================
function prepare_counters(cfg)
% Set up counters used by the processor files.
global global_info

W = cfg.WEEKS;
global_info.cfg        = cfg;
global_info.STOP_AT    = W;       % GPenSIM stops after the last week
global_info.DELTA_TIME = 0.05;    % GPenSIM clock step (all firing times are multiples of 0.05)

% Counters for each week
global_info.n.rain     = zeros(1, W);   % water tokens made by tRain
global_info.n.stored   = zeros(1, W);   % tokens taken into the dam
global_info.n.spilled  = zeros(1, W);   % tokens spilled
global_info.n.released = zeros(1, W);   % tokens sent to the turbines
global_info.n.gen      = zeros(1, W);   % energy tokens made

global_info.level  = nan(1, W);     % water in the dam at the end of each week
```

```
global_info.ledger = zeros(W, 5);   % released water by colour:
                                    % spring, summer, autumn, winter, no colour
end



%% ========================================================================
function pni = build_petri_net(cfg)
% Join the modules and set the starting tokens and firing times.

% Three modules and their connector
pns = pnstruct({'pdf_Inflow', 'pdf_Dam', 'pdf_PowerStation', 'pdf_IMC'});

% Starting water and free space
dyn.m0 = {'pStored', cfg.START, ...
          'pFree',   cfg.CAPACITY - cfg.START};

% Firing times are in weeks.
dyn.ft = {'tTurbine',  0.1, ...
          'allothers', 0.05};

% No priorities needed: the inhibitor arc means tStore and tSpill are never
% enabled at the same time.

pni = initialdynamics(pns, dyn);
end



%% ========================================================================
function R = compute_results(R, cfg, sim, info)
% Convert token counts into water and energy values.
n   = info.n;
W   = cfg.WEEKS;
MM3 = cfg.MM3_PER_TOKEN;
GWH = cfg.GWH_PER_TOKEN;

% A week with no dam event keeps the level of the week before.
level = info.level;
previous = cfg.START;
for w = 1:W
    if isnan(level(w)), level(w) = previous; end
    previous = level(w);
end

% Demand that the generated energy did not cover.
unmet = max(0, cfg.plan_demand - n.gen);

% Find the winter weeks (season 4).
week_of_year = mod((1:W) - 1, 52) + 1;
is_winter    = cfg.season_of_week(week_of_year) == 4;

% Save the results in R.
R.cfg        = cfg;
R.scenario   = cfg.scenario;
R.sim        = sim;
R.n          = n;
R.level      = level;
R.ledger     = info.ledger;
R.unmet_week = unmet;

R.inflow_mm3       = sum(n.rain) * MM3;
R.spilled_mm3      = sum(n.spilled) * MM3;
R.gen_gwh          = sum(n.gen) * GWH;
R.unmet_gwh        = sum(unmet) * GWH;
R.unmet_winter_gwh = sum(unmet(is_winter)) * GWH;
R.min_level_pct    = 100 * min(level) / cfg.CAPACITY;
R.end_level_pct    = 100 * level(end) / cfg.CAPACITY;
R.weeks_at_min     = sum(level <= cfg.MIN_LEVEL);
```

```
% Season of the water released in winter.
% Row 1 is winter 1; row 2 is winter 2.
in_year1 = (1:W) <= 52;
in_year2 = (1:W) > 52;
R.winter_mix = [sum(info.ledger(is_winter & in_year1, :), 1);
                sum(info.ledger(is_winter & in_year2, :), 1)];
winter_total       = max(1, sum(R.winter_mix, 2));
R.summer_share_pct = 100 * R.winter_mix(:, 2) ./ winter_total;
end



%% ========================================================================
function check = run_checks(cfg, sim, info, R)
% Check the results.
n = info.n;

% Check the dam capacity.
check.capacity_ok = all(R.level <= cfg.CAPACITY);

% P-invariant: water + free space = capacity.
check.invariant_ok = tokens_in(sim, 'pStored') + tokens_in(sim, 'pFree') == cfg.CAPACITY;

% Water balance inside the dam.
check.dam_balance = cfg.START + sum(n.stored) - sum(n.released) ...
                    - tokens_in(sim, 'pStored');

% All incoming water must be accounted for.
check.water_balance = sum(n.rain) - sum(n.stored) - sum(n.spilled) ...
                      - tokens_in(sim, 'pCatchment') - tokens_in(sim, 'pRunoff');

% Check water released to the turbines.
check.energy_balance = sum(n.released) - sum(n.gen) - tokens_in(sim, 'pToTurbine');

check.all_ok = check.capacity_ok && check.invariant_ok && ...
               check.dam_balance == 0 && check.water_balance == 0 && ...
               check.energy_balance == 0;
end




%% ========================================================================
function print_summary(R)
c = R.cfg;
fprintf('\n=== %s (%d weeks, %.0f s) ===\n', R.scenario, c.WEEKS, R.runtime_s);
fprintf('Water:  inflow %d Mm3, spilled %d Mm3\n', R.inflow_mm3, R.spilled_mm3);
fprintf('Level:  start %.0f %%, lowest %.0f %%, end %.0f %%, weeks at minimum %d\n', ...
        100 * c.START / c.CAPACITY, R.min_level_pct, R.end_level_pct, R.weeks_at_min);
fprintf('Energy: generated %.0f GWh, unmet %.0f GWh (winter %.0f)\n', ...
        R.gen_gwh, R.unmet_gwh, R.unmet_winter_gwh);
fprintf('Water used in winter 1 / winter 2 [spring summer autumn winter no-colour]:\n');
fprintf('        %s  /  %s   -> summer share in winter 2: %.0f %%\n', ...
        mat2str(R.winter_mix(1, :)), mat2str(R.winter_mix(2, :)), R.summer_share_pct(2));
if R.check.all_ok
    verdict = 'ALL OK';
else
    verdict = 'PROBLEM';
end
fprintf('Checks: capacity %d, invariant %d, balances %d/%d/%d  -> %s\n', ...
        R.check.capacity_ok, R.check.invariant_ok, R.check.dam_balance, ...
        R.check.water_balance, R.check.energy_balance, verdict);
end
```

Listing A.2: run_dam.m

```
% Run the dam simulation.
% Pick a scenario below.
% run_dam runs the Petri net.
```

```
clear; clc; close all;



%% Choose the scenario
scenario = 'S1';   % 'S1' normal, 'S2' wet, 'S3' dry,
                   % 'S4' winter maintenance



%% Run the model
settings = dam_config(scenario);
R = run_dam(settings);



%% Token colours: which season's water made the electricity?
% Each released token is counted by its colour (the ledger in COMMON_PRE).
% The turbine keeps the colour, so this is also the colour of the energy tokens.
names = [R.cfg.season_names, {'no colour (already in dam at start)'}];
used  = sum(R.ledger, 1);
fprintf('\nWater tokens released, by colour:\n');
for k = 1:numel(names)
    fprintf('  %-36s %3d\n', names{k}, used(k));
end



%% Draw the graphs
plot_run(R);
```

Listing A.3: main_dam.m

```
% Run each scenario.
% Print results and save the table.



clear; clc; close all;
% Scenarios: normal, wet, dry, and winter maintenance.
scenarios = {'S1', 'S2', 'S3', 'S4'};
results = {};

for i = 1:numel(scenarios)
    R = run_dam(dam_config(scenarios{i}));
    R.sim = [];
    results{end+1} = R; %#ok<SAGROW>
end

% ---- Table -------------------------------------------------------------
if ~exist('results', 'dir'), mkdir('results'); end
% Save the comparison table as a CSV file that can be opened in Excel.
fid = fopen(fullfile('results', 'scenario_results.csv'), 'w');
fprintf(fid,
'Scenario,Spilled_Mm3,Lowest_pct,WeeksAtMin,Generated_GWh,Unmet_GWh,UnmetWinter_GWh,SummerShareWinter2_
pct,ChecksOK\n');
fprintf('\n%-4s %8s %7s %6s %9s %7s %8s %8s %3s\n', 'Scen', ...
        'Spill', 'Low%', 'Wk@min', 'Gen GWh', 'Unmet', 'UnmetW', 'Summer%', 'OK');
for i = 1:numel(results)
    R = results{i};
    v = [R.spilled_mm3, R.min_level_pct, R.weeks_at_min, R.gen_gwh, R.unmet_gwh, ...
         R.unmet_winter_gwh, R.summer_share_pct(2), R.check.all_ok];
    fprintf('%-4s %8.0f %7.1f %6d %9.0f %7.0f %8.0f %8.0f %3d\n', R.scenario, v);
    fprintf(fid, '%s,%.0f,%.1f,%d,%.0f,%.0f,%.0f,%.0f,%d\n', R.scenario, v);
end
fclose(fid);

plot_compare(results, scenarios);
save(fullfile('results', 'all_results.mat'), 'results');
```

Listing A.4: run_all_scenarios.m

```
function [png] = pdf_Inflow()
% Inflow module: rain and melting snow.
%
%      tRain  ---->  pCatchment  ---->  tRunoff  ---->  (pRunoff, in pdf_IMC)
%    makes one      water flowing      hands the water
%    water token    down the valley    to the dam
%
% tRain adds water. MOD_Inflow_PRE sets the amount
% and gives each token a season colour.
% pCatchment holds water in the catchment.
% tRunoff sends water to the dam.


% GPenSIM uses this name to find MOD_Inflow_PRE.m.
png.PN_name = 'Inflow';


% Places in this module
png.set_of_Ps = {'pCatchment'};


% Transitions in this module
png.set_of_Ts = {'tRain', 'tRunoff'};


% Arcs: {from, to, weight}
png.set_of_As = {'tRain',      'pCatchment', 1, ...   % rain fills the catchment
                 'pCatchment', 'tRunoff',    1};     % runoff takes it away


% Ports connect this module to the others.
png.set_of_Ports = {'tRunoff'};
end
```

Listing A.5: pdf_Inflow.m

```
function [png] = pdf_Dam()
% Dam module: the reservoir.
%
% These places show the water and empty space:
% pStored = water in the dam
% pFree = free space in the dam
% Storing water uses a free slot; releasing water frees a slot.
% This should always stay true:
% pStored + pFree = CAPACITY (P-invariant)
%
% Main transitions:
% tStore: move water into the dam
% tSpill: spill water when the dam is full
% tRelease: send water to the turbine
%
% An inhibitor arc links pFree to tSpill.
% Spilling is blocked while there is free space, so it only happens
% when the dam is full (pFree = 0).
%
% The shared-place arcs are defined in pdf_IMC,
% because those places connect different modules.


% GPenSIM uses this name to find MOD_Dam_POST.m.
png.PN_name = 'Dam';


% Places in this module
png.set_of_Ps = {'pStored', 'pFree'};


% Transitions in this module
png.set_of_Ts = {'tStore', 'tSpill', 'tRelease'};


% Arcs: {from, to, weight}
png.set_of_As = { ...
    'pFree',    'tStore',   1, ...   % storing water uses one free slot ...
    'tStore',   'pStored',  1, ...   % ... and adds one token of stored water
    'pStored',  'tRelease', 1, ...   % releasing takes one token of stored water ...
```

```
    'tRelease', 'pFree',    1};      % ... and gives the free slot back

% Inhibitor arc: {place, transition, weight}
% tSpill can fire only when pFree is empty.
png.set_of_Is = {'pFree', 'tSpill', 1};

% Ports connect this module to the others.
png.set_of_Ports = {'tStore', 'tSpill', 'tRelease'};
end
```

Listing A.6: pdf_Dam.m

```
function [png] = pdf_PowerStation()
% PowerStation module with one turbine.
%
%   (pToTurbine)  ---->  tTurbine  ---->  pEnergy
%
% The turbine uses one water token from pToTurbine
% and makes one energy token in pEnergy.
% One energy token = 50.2 GWh.
%
% GPenSIM keeps the input token colour on the output token,
% so energy from summer water is marked summer.
%
% The weekly turbine limit and maintenance are in the release plan (dam_config.m).
% MOD_PowerStation_POST counts each finished run.

% GPenSIM uses this name to find MOD_PowerStation_POST.m.
png.PN_name = 'PowerStation';

% Places in this module
png.set_of_Ps = {'pEnergy'};

% Transitions in this module
png.set_of_Ts = {'tTurbine'};

% Arcs: {from, to, weight}
png.set_of_As = {'tTurbine', 'pEnergy', 1};   % the turbine makes one energy token

% The turbine takes water from outside this module.
png.set_of_Ports = {'tTurbine'};
end
```

Listing A.7: pdf_PowerStation.m

```
function [png] = pdf_IMC()
% Connector between the modules. It contains shared places and arcs.



% pRunoff: water from Inflow to Dam
% pSpilled: water spilled from the dam
% pToTurbine: water sent to the turbine



png.PN_name   = 'IMC';

png.set_of_Ps = {'pRunoff', 'pSpilled', 'pToTurbine'};
png.set_of_Ts = {};

png.set_of_As = {'tRunoff', 'pRunoff', 1, ...
                 'pRunoff', 'tStore', 1,  'pRunoff', 'tSpill', 1, ...
                 'tSpill', 'pSpilled', 1, ...
                 'tRelease', 'pToTurbine', 1, ...
                 'pToTurbine', 'tTurbine', 1};
end
```

Listing A.8: pdf_IMC.m

```
function [fire, transition] = COMMON_PRE(transition)
% Rules for transitions between modules.
% GPenSIM checks these before a port transition fires.
% Rules inside modules are in the MOD_*_PRE files.

global global_info
cfg = global_info.cfg;
wk = week_now();
fire = 1;                                   % default: no extra condition (tRunoff, tSpill, tTurbine)



switch transition.name

    % Store water only if there is free space. Keep its colour.
case 'tStore'   % water enters the dam (the arc from pFree needs free space)
        % Keep the colour of the oldest runoff token.
        % Otherwise the free-space token colour could mix in.
        tok = tokenArrivedEarly('pRunoff', 1);
        transition.selected_tokens = tok(1);
        transition.new_color = get_color('pRunoff', tok(1));
        transition.override  = 1;

    % Set the release amount and record the water colour.
case 'tRelease' % water to the turbines, following the release plan
        % Follow the release plan and keep the minimum water level.
        fire = global_info.n.released(wk) < cfg.plan_release(wk) && ...
               ntokens('pStored') > cfg.MIN_LEVEL;        % keep the minimum level
        if fire
            tok = tokenArrivedEarly('pStored', 1);        % oldest water first
            transition.selected_tokens = tok(1);
            % Record which season the released water came from.
            k = find(ismember(cfg.season_names, get_color('pStored', tok(1))), 1);
            if isempty(k), k = 5; end      % no colour = water already in the dam at start
            global_info.ledger(wk, k)    = global_info.ledger(wk, k) + 1;
            global_info.n.released(wk)   = global_info.n.released(wk) + 1;
        end
end
end
```

Listing A.9: COMMON_PRE.m

```
function [fire, transition] = MOD_Inflow_PRE(transition)
% Rules for water coming into the model.
% Add rain tokens until the weekly target is reached.
% Give each token the current season colour.

global global_info
cfg = global_info.cfg;

% Stop when this week's inflow target is reached.
if strcmp(transition.name, 'tRain')
    [wk, season] = week_now();

fire = global_info.n.rain(wk) < cfg.plan_inflow(wk);
    if fire
        % Keep the season colour on each token.
transition.new_color = cfg.season_names{season};   % e.g. 'summer'
        transition.override  = 1;
        global_info.n.rain(wk) = global_info.n.rain(wk) + 1;
    end
else
    fire = 1;       % tRunoff: its interface rule is in COMMON_PRE
end
end
```

Listing A.10: MOD_Inflow_PRE.m

```
function [] = MOD_Dam_POST(transition)
% Runs after a Dam transition finishes.
% Count the action and save the dam level for this week.
% The last value saved in a week is the level at the end of that week.

global global_info
wk = week_now();

switch transition.name
    case 'tStore', global_info.n.stored(wk)  = global_info.n.stored(wk) + 1;
    case 'tSpill', global_info.n.spilled(wk) = global_info.n.spilled(wk) + 1;
end
global_info.level(wk) = ntokens('pStored');
end
```

Listing A.11: MOD_Dam_POST.m

```
function [] = MOD_PowerStation_POST(transition) %#ok<INUSD>
% Runs when the turbine finishes.

global global_info
wk = week_now();
% Count the energy made this week.
global_info.n.gen(wk) = global_info.n.gen(wk) + 1;
end
```

Listing A.12: MOD_PowerStation_POST.m

```
function [wk, season] = week_now()
% Find the current week and season.
% One time unit is one week.
% Seasons: 1 spring, 2 summer, 3 autumn, 4 winter.

global global_info

cfg = global_info.cfg;
% Convert simulation time to a week number.
wk = min(floor(current_time() + 1e-6) + 1, cfg.WEEKS);
% Get the season for that week.
season = cfg.season_of_week(mod(wk - 1, 52) + 1);
end
```

Listing A.13: week_now.m

```
function k = tokens_in(sim, place_name)
% Count the tokens left in a place at the end of a run.
names = {sim.global_places.name};
k = sim.X(strcmp(names, place_name));
end
```

Listing A.14: tokens_in.m

```
% Tests for the dam model.
% Each test has a purpose, expected result and check.
% The table at the end shows the expected and actual results.

clear; clc;
results = {};    % one row per test: {name, expected, got, pass}



%% T1  Overflow ------------------------------------------------------------
% Check that water spills only when the dam is full.
% Expected: 4 tokens arrive, but the dam can hold only 3.
% The first 3 are stored and the last one spills.
% Expected result: stored 3, spilled 1, free 0.
R = run_dam(dam_config('T_OVERFLOW'), false);

stored  = tokens_in(R.sim, 'pStored');
```

```
spilled = tokens_in(R.sim, 'pSpilled');
free    = tokens_in(R.sim, 'pFree');

pass = stored == 3 && spilled == 1 && free == 0 && R.check.all_ok;
got  = sprintf('stored %d, spilled %d, free %d', stored, spilled, free);
results(end+1, :) = {'T1 overflow', 'stored 3, spilled 1, free 0', got, pass};



%% T2  Colours, oldest water first --------------------------------------
% Check the season colours and release order.

% Two spring tokens arrive in week 1 and two summer tokens in week 10.
% Three tokens are released in week 12, oldest first.
% Two spring tokens and one summer token leave; one summer token stays.
% Expected release count: [2 1 0 0 0].
R = run_dam(dam_config('T_COLOUR'), false);

released = sum(R.ledger, 1);
left     = R.sim.global_places(strcmp({R.sim.global_places.name}, 'pStored')).token_bank;
left_colour = '';
if numel(left) == 1
    left_colour = strjoin(left(1).color, ',');
end

pass = isequal(released, [2 1 0 0 0]) && numel(left) == 1 && strcmp(left_colour, 'summer');
got  = sprintf('released %s, %d left: %s', mat2str(released), numel(left), left_colour);
results(end+1, :) = {'T2 colours, oldest first', 'released [2 1 0 0 0], 1 left: summer', got, pass};



%% T3  Normal two years: nothing lost or created ------------------------
% Check the balances in a normal run (S1).
% Expected: all balances are zero.
R = run_dam(dam_config('S1'), false);
c = R.check;

pass = c.all_ok;
got  = sprintf('dam/water/energy = %d/%d/%d', c.dam_balance, c.water_balance, c.energy_balance);
results(end+1, :) = {'T3 normal: balances', 'dam/water/energy = 0/0/0', got, pass};



%% T4  Wet years: spill, but never above capacity -----------------------
% Check capacity in a wet year (S2).
% Expected: some water spills when the dam fills,
% but the dam never goes above 155 tokens.
R = run_dam(dam_config('S2'), false);

highest = max(R.level);

pass = R.spilled_mm3 > 0 && highest <= R.cfg.CAPACITY && R.check.all_ok;
got  = sprintf('spilled %d Mm3, highest level %d', R.spilled_mm3, highest);
results(end+1, :) = {'T4 wet: spill, capacity kept', 'spilled > 0, highest level <= 155', got, pass};



%% T5  Dry years: minimum level is kept ---------------------------------
% Check the minimum level in a dry year (S3).
% Expected: the level reaches 16 tokens at least once,
% but does not fall below 16 at the end of a week.
R = run_dam(dam_config('S3'), false);

lowest = min(R.level);

pass = R.weeks_at_min > 0 && lowest >= R.cfg.MIN_LEVEL && R.check.all_ok;
got  = sprintf('weeks at minimum %d, lowest level %d', R.weeks_at_min, lowest);
results(end+1, :) = {'T5 dry: minimum level kept', 'weeks at minimum > 0, lowest level >= 16', got,
pass};
```

```
%% Print the table -------------------------------------------------------
fprintf('\n%-6s %-30s %-42s %s\n', 'Result', 'Test', 'Expected', 'Got');
for i = 1:size(results, 1)
    if results{i, 4}
        verdict = 'PASS';
    else
        verdict = 'FAIL';
    end
    fprintf('%-6s %-30s %-42s %s\n', verdict, results{i, 1}, results{i, 2}, results{i, 3});
end
fprintf('\n%d of %d tests passed\n', sum([results{:, 4}]), size(results, 1));
```

Listing A.15: run_tests.m

```
% Small Petri net for the reachability check.
function [png] = reach_pdf()

% This model uses small fixed values so the number of states stays
% small enough to check.
%
%   pRain --tStore--> pStored --tStart--> pBusy --tGen--> pEnergy
%     |  (needs pFree)          (gives pFree back)  (gives pIdle back)
%     +--tSpill--> pSpilled   (inhibitor arc: only when pFree is empty)

png.PN_name   = 'Dam reachability model';
png.set_of_Ps = {'pRain', 'pFree', 'pStored', 'pSpilled', 'pIdle', 'pBusy', 'pEnergy'};
png.set_of_Ts = {'tStore', 'tSpill', 'tStart', 'tGen'};
png.set_of_As = {'pRain', 'tStore', 1,  'pFree', 'tStore', 1,  'tStore', 'pStored', 1, ...
                 'pRain', 'tSpill', 1,  'tSpill', 'pSpilled', 1, ...
                 'pStored', 'tStart', 1, 'pIdle', 'tStart', 1, ...
                 'tStart', 'pBusy', 1,  'tStart', 'pFree', 1, ...
                 'pBusy', 'tGen', 1,    'tGen', 'pEnergy', 1,  'tGen', 'pIdle', 1};
png.set_of_Is = {'pFree', 'tSpill', 1};
end
```

Listing A.16: reach_pdf.m

```
% Check the states the small Petri net can reach.
% Build the coverability tree for reach_pdf.m.
% Use cotreei because the net has an inhibitor arc.
% Check capacity and invariants.
% Show the P-invariants for both models.

clear; clc;
global global_info
global_info = struct();

% Build the small Petri net.
pns = pnstruct('reach_pdf');
dyn.m0 = {'pRain', 3, 'pFree', 2, 'pIdle', 1};
dyn.ft = {'allothers', 1};          % times are not used by the tree, but GPenSIM asks
pni = initialdynamics(pns, dyn);

fprintf('===== 1. Coverability tree =====\n');
% Run the reachability check.
COTREE = cotreei(pni, 500);         % prints every state

fprintf('\n===== 2. Check every state in the tree =====\n');
% Get place names for the columns in the tree.
P = {pns.global_places.name};
S = COTREE(:, 1:numel(P));          % one row per state, one column per place
col = @(name) S(:, strcmp(P, name));
fprintf('States in the tree:                        %d\n', size(S, 1));
fprintf('Dam never above capacity (pStored <= 2):   %d\n', all(col('pStored') <= 2));
fprintf('Capacity invariant pStored + pFree = 2:    %d\n', all(col('pStored') + col('pFree') == 2));
fprintf('Water invariant (all water places) = 3:    %d\n', ...
        all(col('pRain') + col('pStored') + col('pBusy') + col('pSpilled') + col('pEnergy') == 3));
```

```
fprintf('Turbine invariant pIdle + pBusy = 1:        %d\n', all(col('pIdle') + col('pBusy') == 1));
% Dead (terminal) states are marked with T in the last column of COTREE.
% Check that none of them leaves water waiting in the system.
dead = S(COTREE(:, end) == double('T'), :);
fprintf('Dead states with water still in the dam:    %d  (0 = no deadlock)\n', ...
        sum(dead(:, strcmp(P, 'pRain')) + dead(:, strcmp(P, 'pStored')) + dead(:, strcmp(P, 'pBusy')) >
0));

fprintf('\n===== 3. P-invariants =====\n');
fprintf('Small model:\n');      pinvariant(pns);
fprintf('\nFull Blasjo model:\n');
pinvariant(pnstruct({'pdf_Inflow', 'pdf_Dam', 'pdf_PowerStation', 'pdf_IMC'}));



% ----- Coverability tree diagram -----
% GPenSIM's cotree can draw the tree, but it does not accept nets with
% inhibitor arcs. This net has one (pFree -> tSpill), so cotreei is used
% above: it builds and prints the tree, but cannot draw it.
% The diagram of this tree is in diagrams/03_coverability_tree.png.
% It uses the same state numbers as the cotreei printout:
%   25 nodes, 20 different states
%   D = marking already in the tree (not expanded again)
%   T = dead state (no transition can fire): states 23 and 25
% Both dead states have all the water used up, so there is no deadlock.
```

Listing A.17: reachability_analysis.m

```
% Plot one simulation run.
function plot_run(R)
% Two simple graphs, saved in results/run_<scenario>.png
%   1. How full the dam is each week.
%   2. Which season's water was used in each winter (token colours).

if ~exist('results', 'dir'), mkdir('results'); end
c = R.cfg;

f = figure('Name', R.scenario, 'Color', 'w', 'Position', [100 100 800 600]);

% 1. Dam level
subplot(2, 1, 1);  hold on;
plot(0:c.WEEKS, 100 * [c.START R.level] / c.CAPACITY, 'b', 'LineWidth', 2);
plot([0 c.WEEKS], 100 * c.MIN_LEVEL / c.CAPACITY * [1 1], 'r--');
ylim([0 100]); grid on;
xlim([0 c.WEEKS]);
xlabel('Week');  ylabel('% full');
legend({'dam level', 'minimum level'}, 'Location', 'best');
title(sprintf('Scenario %s: dam level', R.scenario));

% 2. Token colours
subplot(2, 1, 2);
h = bar(R.winter_mix, 0.5, 'stacked');
cols = [0.1 0.7 0.5; 0.9 0.4 0.2; 0.9 0.6 0; 0.2 0.5 0.9; 0.7 0.7 0.7];
for k = 1:5, set(h(k), 'FaceColor', cols(k, :)); end
set(gca, 'XTickLabel', {'Winter 1', 'Winter 2'});
ylabel('Water tokens used');
legend({'spring', 'summer', 'autumn', 'winter', 'old water'}, 'Location', 'eastoutside');
title('Which season''s water was used in winter?');

saveas(f, fullfile('results', sprintf('run_%s.png', R.scenario)));
end
```

Listing A.18: plot_run.m

```
% Compare the scenarios.
function plot_compare(results, scenarios)
% Two simple bar charts, one bar per scenario, value written on each bar.
% Saved in results/compare_scenarios.png

missing = zeros(1, numel(scenarios));
```

```
spilled = zeros(1, numel(scenarios));
for i = 1:numel(scenarios)
    missing(i) = results{i}.unmet_gwh;
    spilled(i) = results{i}.spilled_mm3;
end

f = figure('Name', 'Scenarios', 'Color', 'w', 'Position', [100 100 900 400]);

subplot(1, 2, 1);
simple_bar(missing, scenarios);
title('Electricity missing (GWh)');

subplot(1, 2, 2);
simple_bar(spilled, scenarios);
title('Water spilled (Mm3)');

saveas(f, fullfile('results', 'compare_scenarios.png'));
end



function simple_bar(v, scenarios)
% Bar chart with the number written on top of each bar.
bar(v, 0.6);
ylim([0 1.2 * max([v 1])]);
text(1:numel(v), v, string(round(v)), ...
    'HorizontalAlignment', 'center', 'VerticalAlignment', 'bottom');
set(gca, 'XTickLabel', scenarios);
grid on;
end
```

Listing A.19: plot_compare.m

Appendix B

Table B1: Model constants.

Constant	Value	Meaning
Time unit	1 week	104 weeks simulated, week 1 = early April
MM3_PER_TOKEN	20 Mm^3	Water per token
GWH_PER_TOKEN	50.2 GWh	Energy per token through the turbine
CAPACITY	155 tokens	3,100 Mm^3
MIN_LEVEL	16 tokens	About 10 % full; no release at or below
START	70 tokens	45 % full at the start (S1 and S4)
TURBINE_CAP	4 tokens per week	2 tokens per week during maintenance
DELTA_TIME	0.05 week	Clock step; tTurbine 0.1 week, other transitions 0.05 week
Seasons	weeks 1-9, 10-22, 23-35, 36-52	Spring, summer, autumn, winter